SPFM — Complete Software Requirements

1. Project Overview

SPFM = Shield Project Financial Management

The system is an internal software tool for Shield Security Doors to manage project financial information and bookkeeping-related data.

The main purpose is:

Microsoft Login → Project List → Create/Open Project → Enter/Edit Data → Calculate SPFM → Save → Reopen → Edit → Recalculate

The system should eventually receive much of its financial/project data automatically from QuickBooks, but QuickBooks automatic import is Phase 2 and is NOT included in the accepted $3,000 Growth Plan.

The current goal is to complete a stable Alpha version that works independently first and is structured correctly for the later QuickBooks integration.

2. Existing Starting Point

This is not a project being built completely from scratch.

The client already has:

Backend
ASP.NET Core

Existing API

Existing health endpoint

Swagger

Protected Users/me endpoint

Microsoft Entra authentication already started

Frontend
Next.js

Existing project foundation

Frontend still needs the actual application workflow completed

Repository
Existing private GitHub repository

Developer access already provided

Authentication
Microsoft Entra / Microsoft work account authentication already partially configured

Therefore, the first development task is codebase audit + understanding the existing architecture, not immediately rewriting the application.

3. Main Application Architecture

The application should have these major layers:

Frontend
Next.js application

Backend
ASP.NET Core Web API

Authentication
Microsoft Entra ID + MSAL

Database
Relational database for:

Projects

Project financial information

SPFM inputs

Calculation results

Change history

User information

Future QuickBooks mapping

Future Integration
QuickBooks API

Deployment
Client's internal hosting environment, potentially Azure or another environment selected during deployment.

4. User Type

At launch:

Single internal user

The client specifically stated:

"I am the only user."

Therefore, the Alpha does not need a complicated multi-role permission system.

However, the architecture should not prevent roles from being added later.

Future possibilities:

Admin

Project Manager

Accountant

Staff

Read-only user

But these are future features, not current scope.

5. Authentication

Microsoft Login
The user should be able to access the application using their Microsoft work account.

Required functionality
Microsoft Sign In

MSAL integration

Microsoft Entra authentication

Secure token handling

Access token attached to API requests

Protected API routes

Session persistence

Logout

Authentication error handling

Unauthorized access handling

Security requirements
No:

Client secrets inside frontend code

Hardcoded credentials

Exposed API keys

Sensitive authentication information committed to GitHub

Environment variables should be used for configuration.

6. Application Home / Dashboard

After login, the user should land on a simple internal dashboard.

The dashboard should provide access to the main project workflow.

Dashboard could contain

Projects
Total projects

Recently updated projects

Search

Open project

Create new project

Potential future:

Financial summary

Recent activity

QuickBooks sync status

Warnings

Outstanding actions

The current Alpha should remain focused rather than becoming an unnecessarily complicated ERP dashboard.

7. Project Management

This is one of the most important parts of the application.

The user should be able to:

Create project
Create a new SPFM project.

View projects
Display previously saved projects.

Search projects
Search by relevant project information such as:

Project name

Project number

Customer

Other agreed project identifiers

Open project
Click a project and open its complete information.

Edit project
Modify existing information.

Save project
Persist the updated information to the database.

Recalculate
Run the SPFM calculations again after modifying inputs.

8. Project Creation Workflow

The basic workflow should be:

Login

↓

Projects

↓

New Project

↓

Enter Project Information

↓

Enter SPFM Inputs

↓

Validate Data

↓

Calculate

↓

Display Results

↓

Save

↓

Project Stored in Database

↓

Reopen Later

↓

Edit

↓

Recalculate

↓

Save Again

This complete loop is one of the primary acceptance criteria.

9. Main Project Screen

The main project screen should be the core working interface.

It should contain:

Project information
Relevant project information from the client's SPFM specification.

Financial/project inputs
Fields defined in the handoff document.

Input controls
Depending on the existing specification:

Text inputs

Numbers

Dropdowns

Dates

Selections

Calculated fields

Required fields

Actions
Save

Calculate

Recalculate

Potentially:

Cancel

Back to Projects

10. Form Validation

The application should prevent incorrect/incomplete data from being silently saved.

Examples:

Required field missing

Invalid number

Invalid date

Invalid selection

Negative value where not permitted

Invalid project information

The UI should show understandable error messages.

For example:

Project number is required.

rather than:

ValidationException: NullReference...

11. SPFM Calculation Engine

This is probably the most technically important business-logic component.

The client's existing SPFM formulas, dependencies, selections and calculation rules need to be converted into reliable application logic.

The calculation engine should be separated from the UI.

Conceptually:

User Input

↓

Validation

↓

SPFM Calculation Engine

↓

Calculated Results

↓

Database

This separation is important because the calculations can then be tested independently.

12. Calculation Rules

The development should include:

Existing SPFM formulas

Dependencies between fields

Selection-based rules

Required calculations

Intermediate calculations

Final results

Rounding rules

Numeric precision

Edge cases

The exact formulas should come from the client's handoff document and provided examples rather than being invented during development.

13. Calculation Testing

Critical calculations should have automated tests.

For example:

Input Set A

Expected:

Result A = X

Actual:

Result A = X

Pass

This should be repeated for the important calculation rules.

14. Real Project Validation

The client will provide approximately 2–3 real Shield projects with known expected results.

These should become formal test cases.

For each project:

Test

Load known inputs

↓

Run calculation

↓

Compare output

↓

Verify expected result

↓

Record pass/fail

This is extremely important because financial/business calculations should not be validated only by checking whether the UI "looks correct."

15. Database

The application needs persistent storage.

At minimum, the database architecture should support:

Users
User ID

Microsoft identity information

Email

Name

Created/updated information

Projects
Project ID

Project identifiers

Project information

Status if required

Created date

Updated date

Created by

Updated by

SPFM Data
The actual fields required by the SPFM specification.

Calculation Results
Calculated values where appropriate.

Change History
Records of important project modifications.

QuickBooks Mapping
Even though QuickBooks import is Phase 2, the data model should be structured so QuickBooks information can be mapped later without rebuilding the whole system.

16. Data Persistence

The following must work:

Create
Create a project.

Save
Save the project's data.

Reopen
Close the application and open it later.

Retrieve
All saved information should be loaded correctly.

Edit
Change project information.

Recalculate
Run calculations again.

Update
Save the new values.

The data must survive browser refreshes, logout/login and application restarts.

17. Change History / Audit Trail

The accepted Growth Plan specifically includes:

Change history showing who edited what, and when.

Therefore, this should be part of the Alpha.

The system should record relevant changes such as:

User

Project

Field/data changed

Previous value

New value

Timestamp

For example:

Updesh changed Project X

Estimated Material Cost

$12,000 → $13,500

October 8, 2026, 14:32

The exact level of field-level tracking should be aligned with what the existing code/handoff supports.

18. QuickBooks Integration

This needs to be handled carefully.

The client explicitly said:

Most of the data will automatically be imported from QuickBooks.

And the requirements form says:

Integration with QuickBooks

However, the accepted Growth Plan explicitly excludes the QuickBooks import.

Therefore:

Phase 1

Prepare the application/data model for QuickBooks.

Phase 2
Implement:

QuickBooks → API → Mapping → SPFM Database → Application

Potential future functionality:

Secure QuickBooks OAuth

Connect QuickBooks account

Import projects

Import customers

Import financial information

Map QuickBooks fields to SPFM fields

Synchronize data

Handle duplicate records

Handle changed records

Sync errors

Manual resync

Sync status

Last synchronization timestamp

This should be quoted separately.

19. QuickBooks-Ready Data Architecture

Even though the integration isn't being built now, the database should be designed with future mapping in mind.

For example:

QuickBooks Customer

↓

Customer Mapping

↓

SPFM Customer/Project

↓

SPFM Calculation

This prevents the Phase 2 integration from requiring a complete rewrite.

20. API Architecture

The ASP.NET Core backend should expose secure APIs for the frontend.

Likely API groups:

Authentication
Microsoft authentication validation.

Users
Current user

User profile

Projects
Create

List

Get

Update

Search

Calculations
Calculate project

Recalculate project

Return results

History
Project change history

Future:

QuickBooks
Connect

Sync

Import

Mapping

Sync status

21. API Security

All sensitive application APIs should require authentication.

For example:

GET /api/projects

POST /api/projects

GET /api/projects/{id}

PUT /api/projects/{id}

POST /api/projects/{id}/calculate

should not be publicly accessible.

The API should verify the authenticated Microsoft user before processing requests.

22. Error Handling

The application should handle errors professionally.

Frontend
Instead of crashing:

Something went wrong.

Show something useful:

We couldn't save this project. Please try again.

Backend
Errors should be:

Logged appropriately

Returned using consistent API responses

Not expose sensitive server information

Authentication
Handle:

Expired session

Unauthorized request

Microsoft login failure

Invalid token

Database
Handle:

Connection failures

Save failures

Missing records

Duplicate records

23. Loading States

Because this is a real business application, every asynchronous operation should have proper UX.

Examples:

Saving...

Calculating...

Loading project...

Signing in...

Updating...

The user should not be left wondering whether the application is working.

24. Unsaved Changes

The application should ideally warn the user when leaving a project with unsaved changes.

For example:

You have unsaved changes. Are you sure you want to leave?

This is particularly important for financial/project data.

25. Responsive Design

This is an internal business application, so the priority should be:

Primary
Desktop

Secondary
Laptop

Basic support
Tablet

Mobile does not need to be the primary experience unless the client later requests it.

The interface should nevertheless avoid breaking on smaller screens.

26. UI/UX Design

The client specifically said this is for internal use and has no branding/reference design.

Therefore, the developer should establish a clean internal business-software design.

Recommended characteristics:

Professional

Minimal

Easy to understand

Data-focused

Clear forms

Clear financial results

Strong visual hierarchy

Consistent buttons

Tables where appropriate

Clear validation

No unnecessary animations

The goal is productivity rather than marketing aesthetics.

27. Project Search

The project list should allow the user to quickly locate existing projects.

Possible functionality:

Search

Sorting

Recent projects

Alphabetical sorting

Project number

Customer/project identifier

Advanced filters can be added later if needed.

28. Project Status

If the handoff specification defines project states, those should be implemented.

Potential examples:

Draft

Active

Completed

Archived

However, don't invent status logic that isn't defined in the client's specification.

29. Financial Data Integrity

Because this application handles bookkeeping/project financial information, data integrity is extremely important.

The application should:

Validate numeric inputs

Avoid accidental data loss

Preserve saved records

Use appropriate decimal/financial data types

Avoid floating-point calculation errors where inappropriate

Keep calculation logic deterministic

Record important changes

Money should not simply be handled as arbitrary JavaScript floating-point numbers throughout the system.

30. Database Migrations

Database changes should be reproducible.

The project should include:

Migration files

Database setup instructions

Environment configuration

Seed/setup instructions if required

A future developer should be able to clone the repository and understand how to recreate the database.

31. Environment Configuration

Development and production configuration should be separated.

Potential environment variables:

DATABASE_CONNECTION

API_URL

MICROSOFT_CLIENT_ID

MICROSOFT_TENANT_ID

MICROSOFT_REDIRECT_URI

Actual secret values should never be committed to GitHub.

32. Testing

Testing should happen at several levels.

Unit testing
Especially:

SPFM calculations

Validation

Business rules

API testing
Authentication

Project creation

Project retrieval

Project update

Calculation endpoints

Frontend testing
Login

Project list

New project

Form validation

Calculation

Save

Reopen

End-to-end testing
The complete workflow:

Login → New Project → Enter Data → Calculate → Save → Close → Reopen → Edit → Recalculate → Save

33. Real Project Test Cases

The 2–3 real Shield projects provided by the client should be converted into named test cases.

Example:

TEST-SPFM-001

Project: Shield Project A

Expected Result: X

Actual Result: X

Status: PASS

These can remain as regression tests for future development.

34. Second Testing Pass

The Growth Plan specifically includes a second full testing pass.

This should happen after the main functionality is implemented and initial bugs are fixed.

The second pass should verify:

Authentication

Database

Project workflow

Calculations

History

Validation

Error handling

Deployment

35. Rollback Plan

The accepted offer includes a rollback plan.

This should cover what happens if a deployment introduces a critical problem.

Potential components:

Database backup

Previous application version

Deployment versioning

Restore procedure

Migration rollback where applicable

The exact mechanism depends on the final hosting environment.

36. Deployment

The Alpha should be deployed to the client's internal hosting environment.

Potential environment:

Microsoft Azure

or another environment agreed upon with the client.

Deployment should include:

Frontend

Backend

Database

Environment variables

Microsoft Entra configuration

HTTPS

Production configuration

37. Production Testing

Testing should not stop after deployment.

The live environment should be tested again.

For example:

Production

↓

Login

↓

Create Project

↓

Enter Data

↓

Calculate

↓

Save

↓

Reopen

↓

Edit

↓

Recalculate

↓

Save

Everything should work without requiring the developer's local machine.

38. Documentation

The final repository should include an updated README.

It should explain:

Installation
How to install dependencies.

Development
How to run frontend/backend locally.

Database
How to configure and migrate the database.

Environment variables

What variables are required.

Authentication
How Microsoft Entra configuration works.

Deployment
How to deploy the application.

Architecture
Short explanation of frontend/backend/database.

39. Handover

The client should receive:

Source code

GitHub repository

Database structure

Migration files

Environment configuration instructions

Microsoft Entra setup instructions

Deployment instructions

Test cases

Known limitations

Future development recommendations

No secrets should be included in documentation.

40. Support

The accepted offer says:

2 months of priority support

Support should primarily cover:

Bugs related to delivered functionality

Small adjustments

Deployment assistance

Clarification of delivered features

Alpha stabilization

Large new functionality should be treated as new development.

41. What Is Included in the Accepted Growth Plan

This is the most important scope boundary.

Included
Repository review

Existing codebase assessment

Microsoft frontend login with MSAL

Microsoft Entra integration with existing API

Project data model

Database tables

Project API

Project creation

Project listing

Project search

Project opening

Project editing

Project saving

SPFM calculations

Automated calculation tests

Reopen/edit/recalculate workflow

2–3 real Shield project validation

Named test cases

Change history

QuickBooks-ready data structure

Second testing pass

Rollback plan

Internal deployment

README/setup documentation

2 months priority support

42. Explicitly NOT Included in Current Phase

These should not accidentally become part of the $3,000 scope.

QuickBooks automatic import
This is Phase 2.

User roles
Client currently has one user.

Advanced permission management
Not currently required.

Data exports
Explicitly excluded from the offer.

Advanced error monitoring
Explicitly excluded from the offer.

Complex accounting functionality beyond the agreed SPFM calculations
Not automatically included.

Payment system
This is internal software, not an e-commerce platform.

Customer-facing website
Not required.

Marketing pages
Not required.

Mobile application
Not required.

43. Future Phase 2

After the Alpha is stable, the logical next phase would be:

QuickBooks Integration

QuickBooks

↓

Authentication

↓

Data Import

↓

Data Mapping

↓

SPFM Database

↓

Calculation Engine

↓

Project Financial Management

Potentially:

Automatic synchronization

Manual synchronization

Sync history

Import status

Duplicate handling

Data conflict handling

QuickBooks error handling

Scheduled sync

Financial data reconciliation

This should be quoted separately.

44. Future Expansion Possibilities

Once the Alpha is proven, the application could eventually become a much larger internal financial/project platform.

Potential future features:

User management
Admin

Manager

Accountant

Staff

Permissions
View

Create

Edit

Approve

Export

Reporting
Project financial reports

Monthly reports

Profitability

Cost analysis

Exports
Excel

CSV

PDF

Accounting
Accountant workspace

Financial document preparation

Reconciliation

Advanced analytics
Project profitability

Revenue

Expenses

Margins

Trends

Monitoring
Error monitoring

Application health

API monitoring

45. Recommended Screen Structure

Unlike the Fiverr order form's generic "10 pages", this should be treated as a software application with screens/routes.

A practical structure is:

Authentication
Microsoft Login

Authentication callback/error

Main Application
Dashboard / Projects

New Project

Project Details / Edit

Calculation Results

Change History

User/Profile

Some of these can technically be the same route or component rather than separate pages.

So don't tell the client this is literally a 10-page website. It's a dynamic internal web application.

46. Complete User Journey

The final Alpha should essentially demonstrate this:

Microsoft Login

↓

Application Dashboard

↓

Projects

↓

Search / Select Project

↓

Open Project

↓

Edit SPFM Information

↓

Validate

↓

Calculate

↓

View Results

↓

Save

↓

Change History Recorded

↓

Close / Logout

↓

Login Again

↓

Reopen Project

↓

Edit Information

↓

Recalculate

↓

Save

That is the core Alpha acceptance workflow.

47. Most Important Technical Priorities

If you're actually developing this, I would prioritize the work in this order:

Priority 1 — Understand existing code
Repository → backend → frontend → database → authentication.

Priority 2 — Authentication
Get Microsoft login working end-to-end.

Priority 3 — Database
Build the correct project data model.

Priority 4 — Project CRUD
Create → list → search → open → edit → save.

Priority 5 — Calculation engine
Implement and isolate the SPFM business logic.

Priority 6 — Calculation validation
Use the client's real project examples.

Priority 7 — Change history
Track important modifications.

Priority 8 — Full workflow testing
Login → project → calculation → save → reopen → edit → recalculate.

Priority 9 — Deployment
Get the complete application working on production/internal hosting.

Priority 10 — Documentation + handover
README, setup, database, deployment, test cases.

48. One Important Scope Warning

There are three things you should keep very clear with Samuel:

1. QuickBooks

He expects QuickBooks to eventually automate much of the data entry, but the actual QuickBooks integration is not included in the current $3,000 Growth Plan.

The current phase only makes the system QuickBooks-ready.

2. Fiverr's generic order fields

The Fiverr order showing:

10 pages

50 products

E-commerce

Payment integration

Opt-in form

Autoresponder

should not be used to define this project's actual requirements.

Those are generic Fiverr order fields. The real scope is the custom SPFM software application you agreed with the client.

3. "Following ChatGPT prompts"

The client said most of the design was created in ChatGPT and that coding is largely following prompts/debugging.

That doesn't mean the implementation is simply copying prompts. You still need to validate:

Architecture → security → database → calculations → API → authentication → testing → deployment.

For financial software, especially, the calculation and data integrity need proper engineering and testing.

Final definition

In one sentence, I would define the project internally as:

SPFM is a secure internal project financial management web application built on the client's existing ASP.NET Core + Next.js codebase, allowing an authenticated Shield user to create, manage, calculate, save, reopen and audit project financial data, with the architecture prepared for a separate QuickBooks integration phase.

And the Alpha success condition is:

A Shield user can securely log in with Microsoft, create or open a project, enter valid SPFM data, run accurate calculations, save the project, reopen it later, edit it, recalculate it, and see the relevant change history on the deployed production environment.

This is the scope I'd use as your master developer requirements document before you start coding, so you don't accidentally absorb QuickBooks, roles, exports, monitoring, or other Phase 2 work into the current $3,000 project.